

Estado del producto

Qué hace DevUP hoy y qué le falta. Escrito a partir del código —rutas, migraciones y pantallas contadas una a una—, no de lo que los planes prometían.

Fecha: 18 de agosto de 2026 · rama `docs/traspaso-busqueda-boveda-github-spotify`

En una línea: el espacio de trabajo y el control de ventas están completos y en producción; la tercera promesa —infraestructura y agentes— está empezada, con la bóveda y dos conectores hechos y el resto sin abrir.

De un vistazo

Endpoints de API	119 en 18 módulos
Tablas en base	42 , con RLS en todas
Migraciones	20
Pantallas	16
Comprobaciones de aislamiento	156 , en verde
Comprobaciones del socket del mundo	13 , en verde
Integración continua	<code>ci.yml</code> : tipos, migraciones, aislamiento, mundo y build
Guiones de navegador	13 en <code>e2e/</code> , manuales — fuera de la integración continua

1. Lo que funciona hoy

Espacio de trabajo — completo

Organizaciones, espacios, canales, miembros y roles (propietario, admin, miembro). Invitaciones por correo con caducidad; verificación de cuenta y recuperación de contraseña.

- **Mensajería** por canal, con edición, borrado y marcado de leído.
- **Voz y vídeo** en malla WebRTC, con cifrado extremo a extremo (decisión

0001), pantalla compartida, y **llamada persistente**: cambiar de pantalla no cuelga.

- **Grabación** de llamadas con consentimiento explícito de cada participante.
- **Archivos** con subida directa al almacén en tres pasos (reservar, subir,

confirmar), etiquetas, previsualización y descarga firmada.

- **Tareas** en tablero por columnas, con etiquetas.

- **Notificaciones** con bandeja y marcado.
- **Búsqueda global** sobre mensajes, archivos, tareas, clientes, servicios y

oportunidades, desde un solo sitio en vez de por espacio.

Control de ventas — completo

Servicios, clientes, embudo de oportunidades, cotizaciones con líneas editables, y objetivos que avanzan solos con las ventas.

El dinero va en **céntimos enteros de punta a punta**; solo se divide para mostrarlo.

Vista inmersiva (DevVerse) — funcional, en beta

El espacio recorrible con avatares, salas, zonas y muebles. Reparto de presencia por zonas para que la malla de voz no pida una conexión por cada persona de la organización. Editor de escenarios.

Tiene sus propios 13 casos de prueba del socket.

Infraestructura y agentes — empezada

- **Bóveda de credenciales:** `connections` + `connection_secrets`, cifrado

AES-256-GCM con una clave maestra propia, separada de la de sesiones.

- **Conector de GitHub:** repos con commits recientes, PRs e issues abiertas,

estado de la última ejecución de CI, refrescado cada 10 minutos.

- **Música compartida de Spotify:** reproducción real por canal, cola

compartida, y el estado repartido por el mismo socket que los mensajes.

- **Entorno de desarrollo embebido** (`/dev`): editor Monaco y terminal real

sobre WebContainer. Fase 0.

Personalización

- **Panel personal** en rejilla: cada tarjeta se arrastra y se redimensiona, y

se guarda por persona. Nadie más en la organización puede verlo ni tocarlo.

- **Noticias** de la organización y **enlaces fijados**.
- **Foto de la organización**.

2. Lo que está a medias, y en qué punto

Spotify: funciona, pero con un techo que no es nuestro. La aplicación está en *modo desarrollo* en el panel de Spotify, y eso impone dos límites que **no se arreglan con código**:

- Solo **5 cuentas** además del dueño pueden usarla. Hoy la lista está vacía, así

que a cualquier compañero le falla todo con «The user is not registered for this application».

- Las **canciones de una lista no se pueden leer** (403 en

`/playlists/{id}/tracks`, también con `fields`, con `market` y pidiendo la lista entera). Se dejó de intentar a propósito. Poner una lista entera sí funciona, porque va por `context_uri`.

La salida de ambos es la misma: pedir la extensión de cuota en el panel de Spotify.

Entorno de dev (`/dev`): Fase 0 y con una condición frágil. Necesita aislamiento cross-origin, que ahora está acotado a su ruta porque aplicado a todo el sitio impedía que el reproductor de Spotify arrancase. Eso obliga a entrar por navegación dura (`<a>`, no `<Link>`). **Si alguien lo convierte en `<Link>`, el entorno deja de funcionar** y no es evidente por qué.

Conector de GitHub: falta explicar sus 404. Un 404 al añadir un repo casi siempre es que el token de alcance fino no lo tiene autorizado, o que la organización bloquea esos tokens. No es un fallo de DevUP, pero la interfaz no lo dice y parece uno.

3. Lo que falta

Lo que cierra la tercera promesa

- **Vista unificada de infraestructura** (S7, ~22 puntos): entornos y despliegues

del cliente en una sola pantalla. Es también lo que desbloquea los muebles vivos que siguen sin conectar en DevVerse — la pantalla de despliegue y el rack de servidores están puestos y no hacen nada.

- **Base de datos como código** y migraciones sincronizadas con el repositorio

(S8–S9).

- **Agentes** (Codex, Claude Code) sobre el entorno embebido (S10).
- **DevUP ID** y la beta (S11–S12).

Detalle en `docs/DevUP-Plan-de-Desarrollo.pdf`.

Calidad e infraestructura propia

- **La integración continua no llega hasta el final.** `ci.yml` corre en cada

push y cada PR —tipos, migraciones, aislamiento entre organizaciones, sala del mundo, migraciones idempotentes y build—, que es la parte difícil y está bien cubierta. Lo que queda fuera son los **13 guiones de `e2e/`**, que siguen lanzándose a mano: son justo los que ejercitan el navegador de punta a punta, y ahí es donde se esconden los fallos que ninguna prueba de unidad ve. El bucket del almacén que nunca se creaba es el ejemplo: el typecheck no podía cazarlo, una subida de verdad sí.

- **El despliegue automático está escrito pero no enchufado.**

`.github/workflows/deploy.yml` espera a que la integración continua termine en verde y despliega en el runner autoalojado de la máquina de producción. El runner está descargado en `C:\Users\Juan\actions-runner` pero **sin configurar**: no hay `.runner`, ni credenciales, ni servicio. Hasta que se registre, los despliegues se quedan en cola y hay que seguir lanzándolos a mano.

- **Redis** para presencia y límite de peticiones, el día que haya más de una instancia de API. Hoy ambos viven en memoria y no sobreviven a un segundo proceso.

- **SMTP real**. Sin él, invitaciones y recuperaciones se escriben en el registro del servidor en vez de enviarse: funciona para probar, no para usar.

- **TURN** para las llamadas. Sin él conectan, pero no se oye nada en NAT simétrico ni en buena parte de las redes móviles.

- **Copias de seguridad**. No hay ninguna definida para Postgres ni para el almacén.

Pendientes acotados

- **Histéresis de tres radios dentro de una sala** (~8 puntos). El reparto por

zonas resuelve veinte personas en cuatro salas; no resuelve doce en una, donde la malla vuelve a pedir once conexiones por cabeza. Descrito en §11 bis de la decisión 0002.

- **Decisiones 0003 y 0004 sin aprobar**: arquitectura de despliegue (monolito

frente a microservicios, VPS con y sin coste) y el conector de GitHub embebido con la semilla de agente.

4. Riesgos y deuda que conviene tener presentes

Un arreglo mejor lleva días sin fusionar. El 415 detrás del túnel está resuelto en main **en el cliente**; en `claude/inicio-desarrollo-nu1ftu` hay una solución **en el servidor** —analizador comodín que acepta cuerpo vacío, más prueba de regresión— que protege a cualquier cliente y no solo al nuestro. La rama también trae 5 pruebas de navegador en Playwright, que son justo el trozo que a la integración continua le falta: hoy cubre tipos, migraciones y aislamiento, pero nada que abra un navegador.

RLS falla en silencio. Una tabla sin política no da error: afecta 0 filas y sigue. Ya pasó una vez —el refresco de tokens de Spotify no persistía nada— y costó una migración (0018) encontrarlo. Toda tabla nueva necesita política **y** un caso en `isolation.test.ts`, y todo UPDATE que importe debe comprobar `rowCount`, no que no haya excepción.

`VAULT_MASTER_KEY` **no se puede perder ni cambiar**. Descifra todas las credenciales de terceros guardadas. Si cambia, todas las conexiones quedan indecifrables para siempre y hay que rehacerlas una a una.

Las subidas estuvieron rotas desde el principio sin que se notara. El bucket del almacén nunca llegó a crearse: la API lo intentaba en cada arranque, MinIO lo rechazaba y el aviso se perdía entre los registros. Como los bytes van directos del navegador al almacén, el fallo era invisible desde el servidor. Arreglado el 18 de agosto. El patrón —un fallo de arranque que solo se manifiesta a mitad de un flujo de tres pasos— es el que conviene vigilar en el resto.

La oficina beta es zona restringida. Instrucción expresa: no se toca `components/world/**`, `lib/world/**`, las rutas de `devverse` ni `lib/view-mode.tsx` sin pedirlo.

5. Cómo comprobar que sigue en pie

```
npm run typecheck && npm run test:rls && npm run test:world
```

Los tres tienen que estar en verde antes de empezar nada. Los guiones de navegador, a mano, están descritos en `e2e/README.md`.

Desplegar:

```
docker compose --env-file .env.production -f docker-compose.prod.yml up -d --build
```

Por dónde seguir leyendo

- `CONTINUAR-AQUI.md` — el informe de estado vivo.
- `CONTEXT0-COMPLETO.md` — arquitectura y el porqué de

cada decisión de la base. 3. `traspaso-2026-08-18-fusion-y-boveda.md` — lo aprendido en la sesión del 17–18 de agosto, con las trampas que ya costaron tiempo. 4. `decisiones/` — lo cerrado, que no se reabre sin motivo nuevo.